Correction: Récursivité + Chaine de caractères

Exercice 1

```
Fonction binaire (n : entier) : caractère
Début
si n < 1 alors
binaire ← 0
sinon
binaire ← binaire (n div 2) + convch(n mod 2)
fin si
 fin
```

Exercice 2

```
Fonction inverse (n : entier) : caractère
Début
si n < 1 alors
inverse ← ""
sinon
inverse ← convch(n mod 10)+inverse (n div 10)
fin si
 fin
```

Exercice 3

```
Algorithme Fibonacci(n : entier) : entier
Début
Si n <= 1 alors
Fibonacci ← 1
Sinon
Fibonacci(n – 1) + Fibonacci(n – 2)
FinSi
fin
```

Exercice 4

```
Fonction chiffre (n : entier, k : entier)
Début
                Si (k = 1) /* le premier chiffre à partir de la
droite */
                     chiffre ← n mod 10 ;
                Sinon
                     chiffre ← chiffre (n div 10, k - 1) ;
Fin
```

Exercice 5

```
Algorithme Ackermann(m, n :entier) : entier
      Début
            si m = 0 et n > 1 alors
                  Ackermann ← n + 1
            sinon
            si n = 0 et m > 0 alors
                  Ackermann ← Ackermann (m– 1,1)
            sinon
                  Ackermann ← Ackermann(m– 1, Ackermann(m, n – 1))fin
            si
            fin si
      fin
```

Exercice 6

```
Fonction somme_recursif (n : Entier) : Entier
      Debut
      Si(n<1) Alors
            somme_recursif ←0
      Sinon
            Si(n=1)Alors
                  somme_recursif ←1
      Sinon
            somme_recursif ←(n + somme_recursif(n-1))
      FinSi
Fin
```

Exercice 7

```
Fonction somme_recursif (n : Entier) : Entier
      Debut
      Si (n<1) Alors
            somme_recursif ←0
      Sinon
            Si(n=1)Alors
                  somme_recursif ←1
      Sinon
            somme_recursif ←(n + somme_recursif(n-2))
      FinSi
Fin
```

Exercice 10

```
Fonction Palindrome (ch : Chaine) : Booleen
     Si Long (ch) < 2 Alors
          Palindrome ← Vrai
     Sinon Si ch [1] = ch [Long(ch)] Alors
          Palindrome ← Palindrome (Sous_chaine (ch, 2, Long(ch)-2))
     Sinon
          Palindrome ← Faux
     FinSi
Fin Palindrome
```

Exercice 11

```
Fonction verif (T1, T2 : tab ; n, m : entier) : booléen
     Si (n<>m) alors
         Verif ←faux
     Sinon si (n=0) et (m=0) alors
       Verif←vari
Sinon
     Si (T1[n]=T2[m]) alors
         Verif ← verif(T1,T2,(n-1),(m-1))Sinon
         Verif ←faux
   Fin si
Fin si
Fin verif.
Fonction occurrence(T : tab ; n, x :entier) :entier
   Si (n=0) alors
      occurrence ←0

  sinon  si (T[i]<>x) alors occurrence
     ← occurrence (T,n-1)
Sinon
    occurrence ←1+ occurrence(T,n-1)
FinSi
Fin occurrence.
Fonction test (T1,T2 :tab ;n, m :entier) :booléen
         Si (n=0) et (m=0) alors
              test←vrai
         Sinon
                 Si (n<>m) alors
                        test←faux
                 Sinon si occurrence (T1, n, T2[m])=0 alors
                          test←faux
                 Sinon
                       test←test(T1,T2,(n-1),m)
               FinSi
    FinSi
```

```
Fin test.
```

Exercice 12

```
Algorithme
Var
n : Entier
Fonction saisie (num : entier) : Entier
Début
Ecrire ("donner un numéro impaire:")
Lire(m)
Si m mod 2=0 Alorssaisie
← saisie ()
Sinon saisie
← mFinSi
Fin
Procédure Etoile (j : Entier)
Début
Si j>=1 Alors
Ecrire ("*")
Etoile (j-1)
Fin Si
Fin

Procédure Espace (i :Entier)
Début
Si i>=1 Alors
Ecrire (" ")
Espace (i-1)
Fin Si
Fin

Procédure Dessine (k, m : Entier)
Début
Si k<=m Alors
Espace (m-k)
Etoile(k)
Etoile (k-1)
Ecrire ("\n")
Fin Si
Fin
{Programme principal}
Début
saisie (num)
Dessine (1, n)
Fin
```

Exercice 13

```
Procédure somme (T1, T2, var T : tab ; n, m : entier)
Début
Si (m=0) et (n<>0) alors
   T[n]←T1[n]
      n←n-1
    somme(T1,T2,T, n,m)
sinon si (n=0) et (m<>0) alors
    T[m]←T2[m]
   m←m-1
  somme(T1,T2,T, n,m)
sinon
  T[n+m]←T2[m]
   m←m-1
somme(T1,T2,T, n, m)
fin si
Fin somme.
```

Exercice 14

```
Procédure addition (T1, T2, var T :tab,n,m :entier)
Début
Si n<>m
     addition ← 0Si
 (n=m) alors
       T[n]←T1[n]+T2[m]
        n←n-1
        m←m-1
     addition (T1, T2, T, n, m)
finsi.
Fin
```

Exercice 15

Ecrire un algorithme qui lit deux mots et qui détermine s'ils sont anagrammes. Sachant qu'un mot est dit anagramme d'un autre mot s'ils utilisent (sont formés par) les même lettres.
Exemples :
CHIEN anagramme de CHINE, NICHE,
AIMER anagramme de MAIRE, MARIE, RAMIE,
GELER n'est pas anagramme d' ALGER, …

Solution : les deux mots puis sont anagrammes si chaque caractère apparait le même nombre de fois dans les deux mots.

```
Algorithme Anagramme
Var M1,M2 :chaine
T1,T2,I,F1,F2 :entier
X :caractère
Anag :booleen
Debut
Ecrire('donner 2 mots')
Lire(M1,M2)
T1←Taille(M1) ; T2←Taille(M2)
Anag←Faux
Si T1=T2
Alors Anag←Vrai ; I←1
Tantque I≤T1 et Anag Faire
X←M1[I] ,F1←0
Pour J de 1 à T1 Faire
Si M1[J]=X Alors
F1←F1+1
Fsi ; Finpour
F2←0
Pour J de1 à T1 Faire
 Si M2[J]=X Alors
F2←F2+1 Fsi ; Finpour
Si F1<>F2 Alors
 Anag←Faux
Fsi
I←I+1
Fintq
Fsi
Si Anag Alors
Ecrire('Les deux mots sont anagrammes')
Sinon
Ecrire('Les deux mots ne sont pas anagrammes')
```

Fsi /Fin.